

NUS-ISS

*Pattern Recognition using
Machine Learning System*

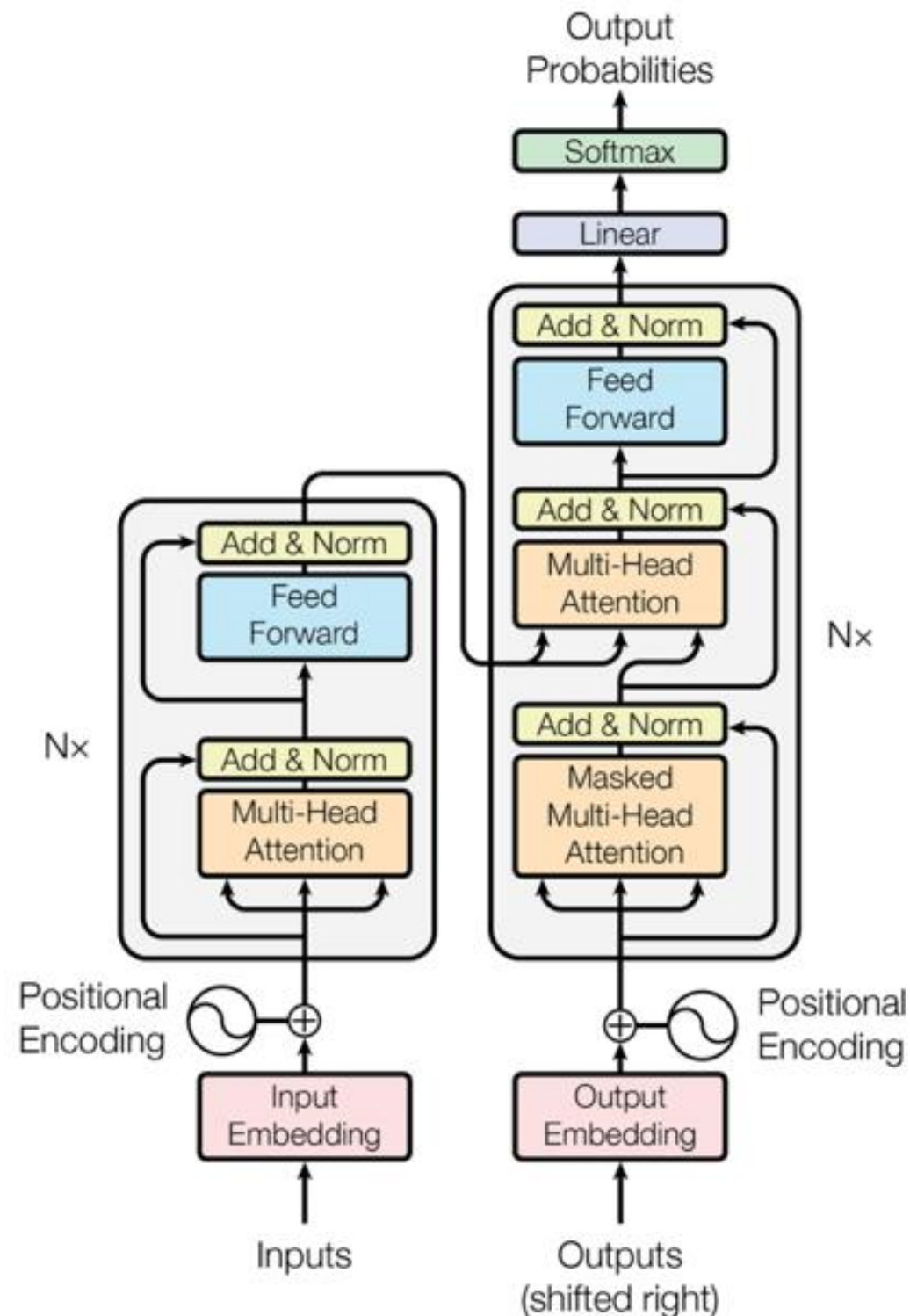


Module 7 - Transformers

by Dr. Gary Leung

© 2026 National University of Singapore.
All Rights Reserved.

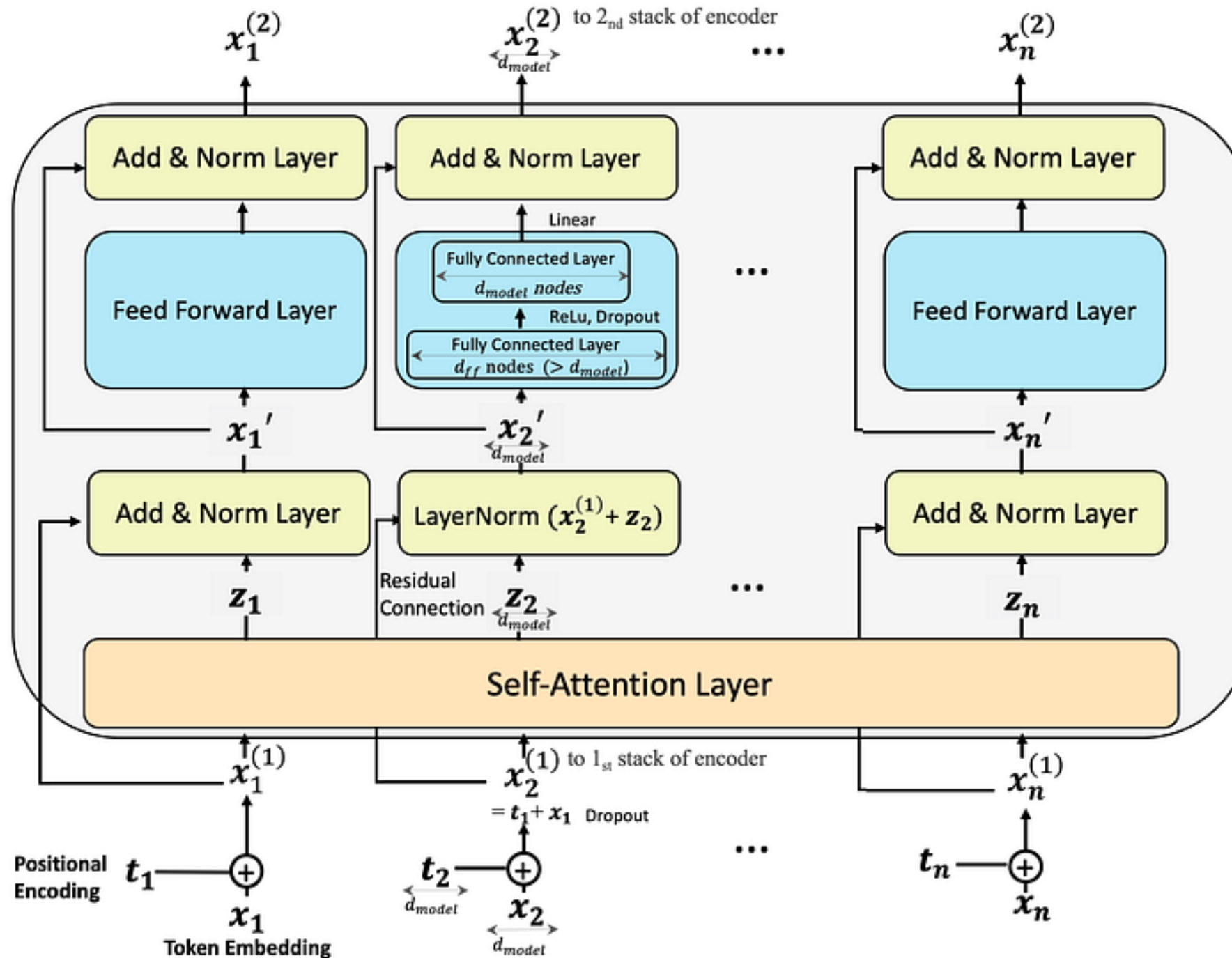
Transformer Architecture



- Originally designed for machine translation
- Later adopted for encoder-based models (e.g., BERT) and decoder-based models (e.g., GPT)
- Left: N stacked decoders process inputs
- Right: N stacked decoders generate outputs

(Image source: [Vaswani, et al., 2017](#))

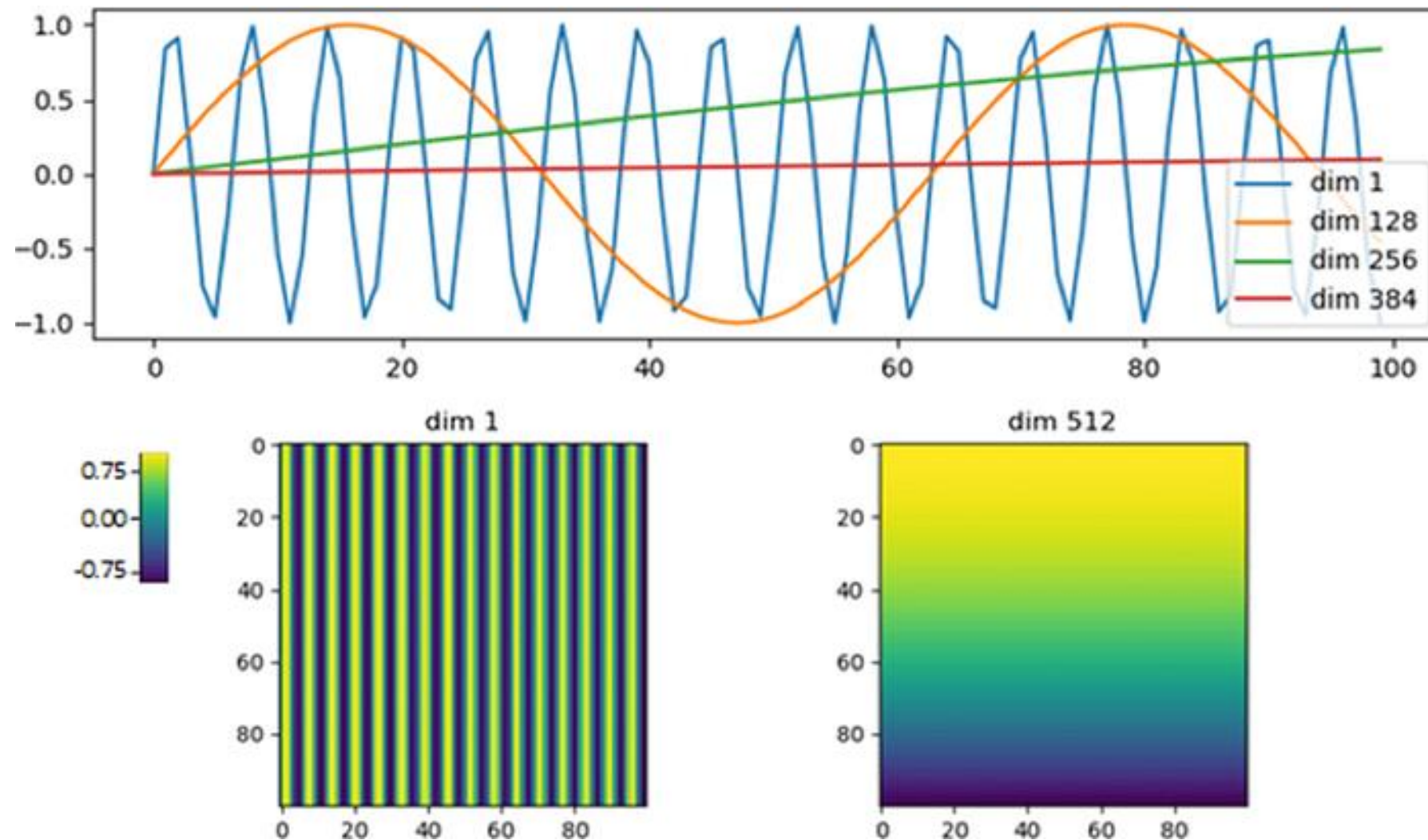
Transformer Encoder



First Stack of Encoder. An encoder stack has four layers: a self-attention layer, two Add & Norm layers and a Feed-Forward layer. (Image source: <https://medium.com/the-modern-scientist/detailed-explanations-of-transformer-step-by-step-dc32d90b3a98>)

Positional Encodings

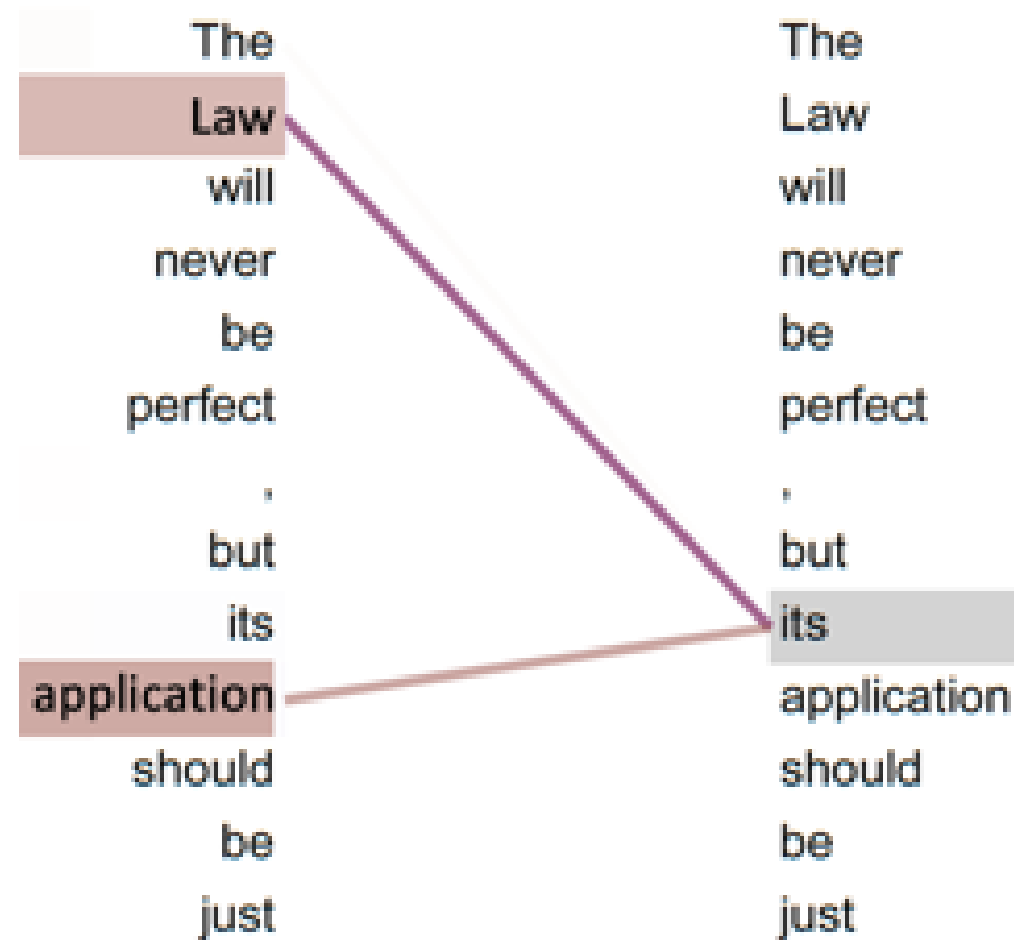
inject sequence order information



Positional encodings for words at different position i across embedding dimensions. *Image Source: [Zelun Wang, et al., 2019](#)*

Self-Attention Layer

calculates attention score between different tokens



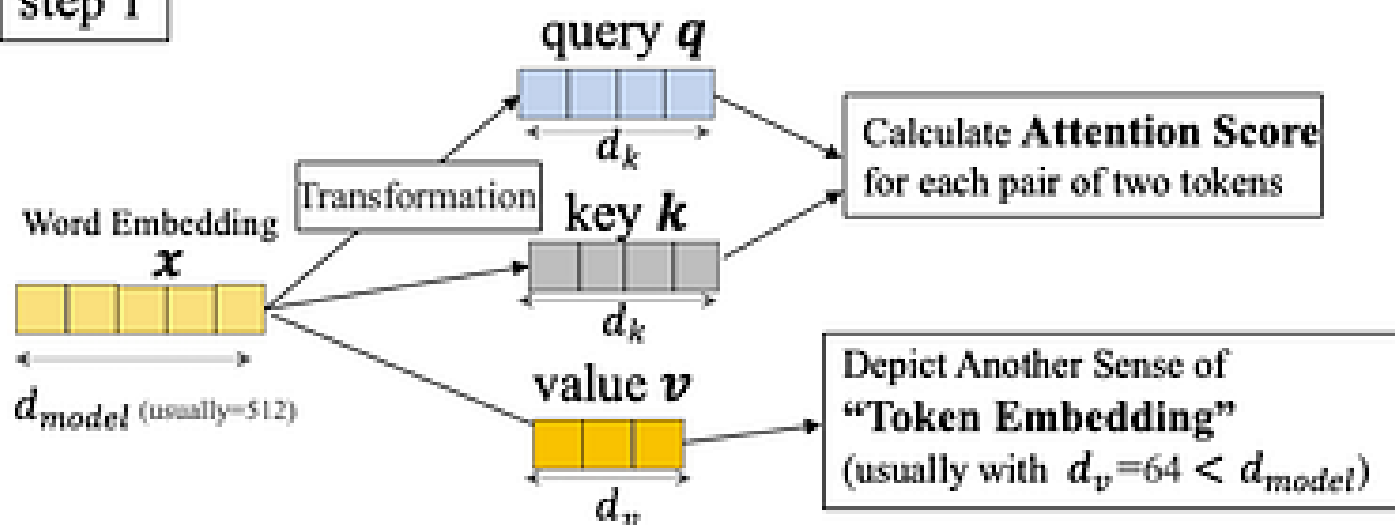
Attention Score of “its” in One Attention Head . The word “its” attends strongly to both “law” and “application”, showing how self-attention captures contextual relationships. (image source: [Vaswani, et al., 2017, revised](#))

One-Head Attention

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

One Head Attention Mechanism

step 1



step 2

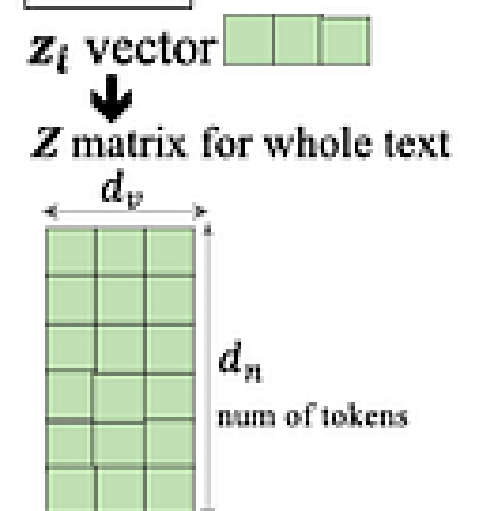
The Attention Score of i^{th} word and the j^{th} (all words)

$$\text{score}_{ij} = \text{softmax} \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)$$

The weighted sum values of all words on i^{th} word

$$\mathbf{z}_i \text{ (dimension } d_v) = \sum_j \text{score}_{ij} \mathbf{v}_j$$

step 3



One-head attention computes how each word attends to all others using Query, Key, and Value vectors. Attention scores ($QK^T / \sqrt{d_k}$) weight the Value vectors, producing context-aware representations.

Image source: <https://medium.com/the-modern-scientist/detailed-explanations-of-transformer-step-by-step-dc32d90b3a98>

Multi-Head Attention

- Different weight matrices W_i^q , W_i^k , W_i^v are learnt in each head
- Different heads capture different types of relationships

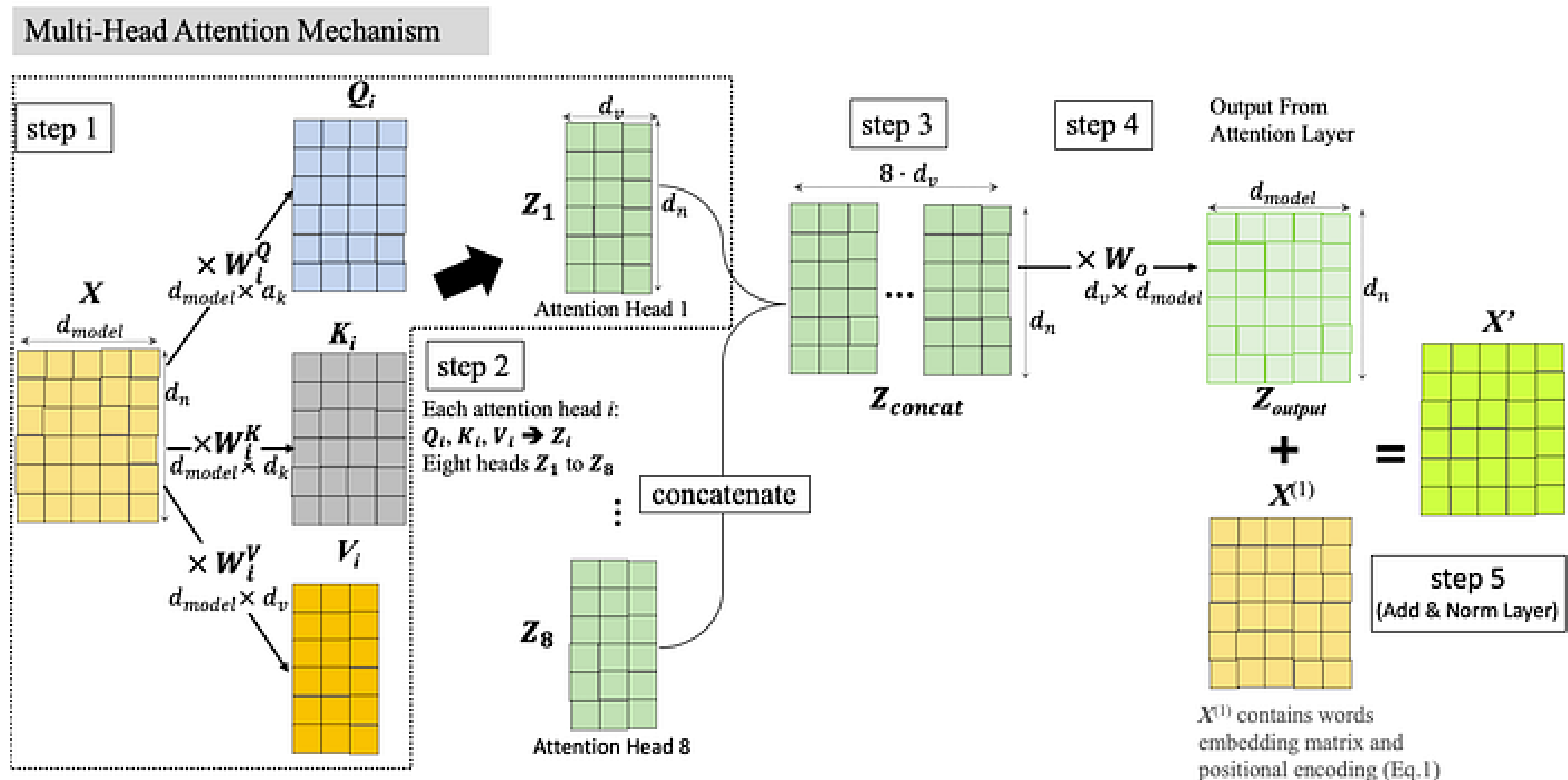
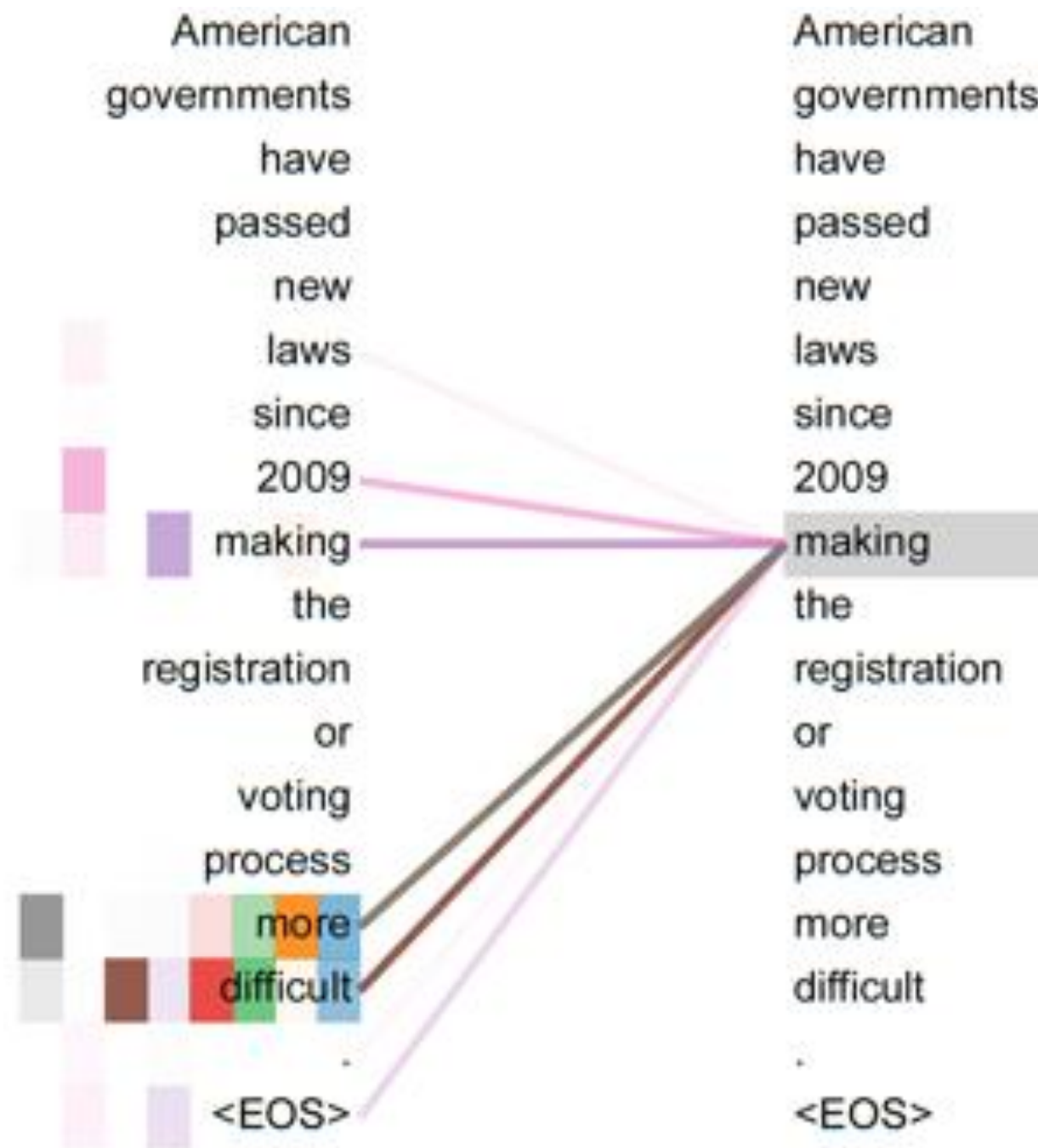


Image source: <https://medium.com/the-modern-scientist/detailed-explanations-of-transformer-step-by-step-dc32d90b3a98>

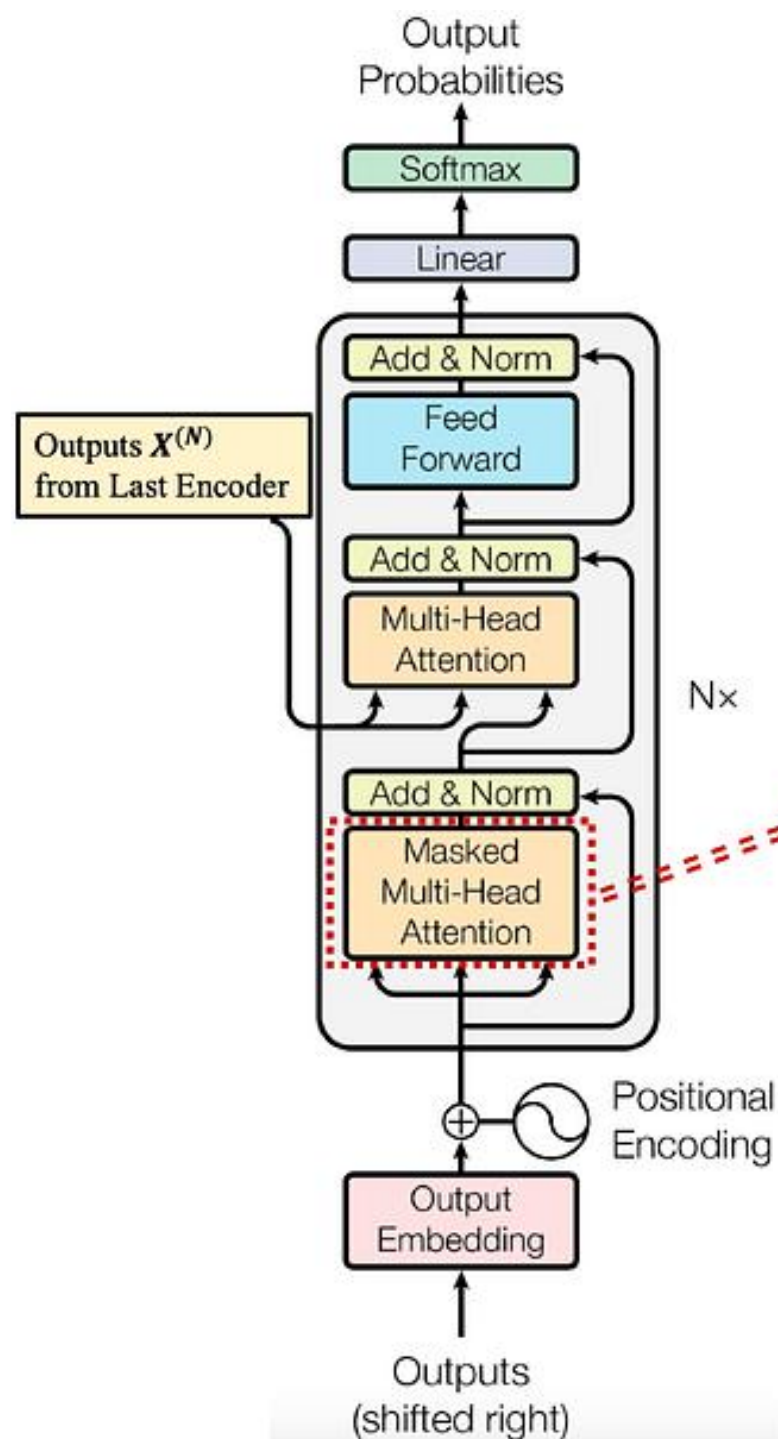
Multi-Head Attention



Attention Scores of “making” in Eight Different Attention Heads. In each head, the word “making” recognizes its relevance with other words in different sense. In head 2, “making” recognizes “2009” is its subject. In other heads, it recognizes “making more difficult” is a phrase. (image source: [Vaswani, et al., 2017](#), revised)

Transformer Decoder

- The Transformer decoder uses masked self-attention to ensure predictions depend only on past tokens.
- Causal masking prevents “cheating” by blocking attention to future positions.
- The decoder also includes cross-attention to the encoder’s output (for tasks like translation).



Scores Matrix

	The	Transformer	is	awesome
The	1	0	0	0
Transformer	0.25	0.75	0	0
is	0.15	0.35	0.45	0
awesome	0.15	0.25	0.2	0.4

Causal Masking

Image Source: <https://medium.com/the-modern-scientist/detailed-explanations-of-transformer-step-by-step-dc32d90b3a98>

Transformer Variants

Encoder-only	Decoder-only	Encoder–Decoder
Processes the entire input sequence at once (bidirectional).	Generates tokens step by step (autoregressive).	Encoder processes the input sequence; decoder generates output using cross-attention to encoder outputs.
Self-attention : tokens attend to all tokens (no masking).	Masked self-attention : tokens attend only to previous tokens (causal masking).	Decoder has two attention layers : (1) masked self-attention and (2) encoder–decoder cross-attention.
Captures context from both left and right .	Captures context only from the left (history).	Captures full context from encoder + autoregressive decoding.
Used for understanding tasks : classification, embeddings, sentence similarity.	Used for generation tasks : text completion, story generation.	Used for sequence-to-sequence tasks : translation, summarization, dialogue.
Example: BERT	Example: GPT	Example: T5, BART